

CSCE 627 THEORY OF COMPUTABILITY · MODULE 11 · LECTURE NOTES

Other Models

The same class, reached many ways.

MODULE OUTCOMES

- Explain the lambda calculus and its equivalence.
- Explain universality in cellular automata.
- Explain minimal Turing-complete systems.
- Explain what quantum computation changes and what it does not.
- Explain hypercomputation proposals and what they assume.

1 The lambda calculus

THE ENTIRE SYNTAX:

x	a variable
$(\lambda x . M)$	a function
$(M N)$	an application

THE ONLY RULE: beta reduction.

$((\lambda x . M) N) \rightarrow M$ with N substituted for x

NO numbers. NO booleans. NO data structures. NO recursion construct. NO state. NO control flow.

AND IT IS TURING COMPLETE.

CHURCH NUMERALS: represent n as the function that applies f n times. $2 = \lambda f x . f (f x)$

BOOLEANS: $\text{true} = \lambda a b . a$; $\text{false} = \lambda a b . b$
and then if-then-else is just application

PAIRS, lists, trees: all encodable as functions

RECURSION: the Y COMBINATOR, which is the recursion theorem's fixed point (Module 09 section 3) made syntactic -- a term Y with $Y f = f (Y f)$

WHY IT MATTERS TO YOU: this is the foundation of every functional language and of the typed core of every language with closures. Your lambda expression is literally this, and the theory behind CSCE 605's type checking is the typed version of it.

Turing completeness from function application alone — which is why 'what is the minimal sufficient mechanism for computation' has such a surprising answer, and why the coincidence of Module 01 §2

is evidence rather than coincidence. **Church and Turing's formalisms could hardly look less alike, and they define the same functions.**

2 Universality from almost nothing

System	What it has	Note
Two-counter machine	Two unbounded counters; increment, decrement, test for zero.	Minsky. Two counters suffice; one does not.
Rule 110	A one-dimensional cellular automaton with a single local rule.	Proved universal by Cook, 2004. No control flow, no memory addressing, no instructions at all.
Conway's Game of Life	A two-dimensional automaton with four rules.	Gliders act as signals, and a full universal machine has been constructed within it.
Tag systems	Delete symbols from the front of a string and append according to a rule.	Post. Two symbols and a deletion number of two suffice.
SKI combinator calculus	Three constants and application.	No variables at all — the lambda calculus with variables eliminated.
Magic: The Gathering	Published card interactions, in a legal game state.	Genuinely proved Turing-complete, 2019, which makes certain questions about a game state undecidable.

Universality is cheap, which is a genuinely useful warning rather than a curiosity: **any sufficiently expressive configuration language, template system, macro facility, or rule engine is probably Turing-complete**, and therefore every interesting question about it is undecidable (Module 06 §2).

ACCIDENTAL TURING COMPLETENESS IS A REAL ENGINEERING HAZARD

If your configuration format, template language, or rule engine turns out to be Turing-complete, then whether a given configuration terminates is undecidable — and you have built a programming language by accident, without a debugger, a profiler, or a specification.

It has happened repeatedly and consequentially: C++ templates (discovered rather than designed), sendmail configuration, spreadsheet formulas with iterative calculation enabled, several build systems, and a number of policy and markup languages.

The symptom is a configuration that hangs, or a build that does not terminate — and no static check can rule it out, by Module 05. The problem cannot be fixed by better tooling, only by changing the language.

So design deliberately for weakness. A non-Turing-complete configuration language can be statically analysed, validated, bounded, and optimised — which is Module 03 §4's weakest-sufficient-model principle with real teeth, and is why formats like Dhall and Starlark are deliberately restricted. **Expressiveness is not free, and the price is paid in analysability.**

3 Quantum computation

QUANTUM COMPUTATION CHANGES THE SPEED, NOT THE CLASS

A quantum computer computes exactly the same functions as a Turing machine. It can be simulated classically, with exponential slowdown, by tracking the amplitudes — so the computable functions are unchanged.

So it does not decide the halting problem, and no undecidability result in this course is affected. Not partially, not in some cases, not in principle — the class of computable functions is identical.

What changes is complexity. BQP, the class of problems solvable in polynomial quantum time, is believed to strictly exceed P — Shor's factoring algorithm is the main evidence — which is CSCE 637 and CSCE 640's subject rather than this one.

And the Church–Turing thesis is untouched, while the *extended* Church–Turing thesis — that every physical process can be *efficiently* simulated classically — is what quantum computing challenges. These are two different claims and conflating them is the usual error: quantum computing is evidence against the second and says nothing about the first.

4 Hypercomputation

- **An oracle. Works, trivially** (Module 07 §1) — **and assumes you have one**, which is the entire question rather than an answer to it. The oracle is a mathematical device for comparing difficulties, not a proposal.
- **Infinitely many steps in finite time** — Zeno machines that halve the time per step, or relativistic computation using time dilation near a rotating black hole. **Requires unbounded energy density, or spacetime geometry nobody has access to.**
- **Infinite precision real numbers as state.** Assumes a physical quantity can carry infinite information, which thermodynamics and the Bekenstein bound both dispute — and which, if granted, makes the result unsurprising.
- **Analog computation.** Looks promising, and **reduces to the previous item once measurement noise is modelled** — the extra power comes entirely from the assumed precision, and vanishes with any finite noise floor.
- **And a true random source. Does not help** — **randomness does not add computability, only speed** (and possibly not even that, which is CSCE 658's derandomisation question). A randomised machine computes the same functions.
- **The pattern is that every proposal assumes an infinity somewhere** — infinite precision, infinite energy, or infinitely many steps compressed into finite time — **and that regularity is a substantial part of why the Church–Turing thesis is robust after ninety years.**

WHY THE AGREEMENT OF THE MODELS IS THE REAL RESULT

Every reasonable definition of computation, proposed for different reasons by different people across ninety years, defines exactly the same class of functions.

And wildly dissimilar systems reach it — pure function application, a one-dimensional cellular automaton, two counters, a string-rewriting system, a trading card game — **which suggests the class is a natural mathematical object rather than an artefact of any particular formalism.**

So the impossibility results of this course are about computation itself, not about Turing machines — **which is what makes them worth a semester** and what makes Module 05's permanence claim defensible.

And the engineering lesson runs the other way round: universality is cheap, so weakness has to be designed for deliberately (§2) if you want a language whose programs you can analyse. The two halves of this module are the same fact read in opposite directions, and the second is the one you will use.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Barendregt — The Lambda Calculus; and Pierce's TAPL chapter 5	§1's calculus , with Pierce's treatment being the right entry point for a programmer.
Cook — Universality in Elementary Cellular Automata (free)	§2's Rule 110 result , which is remarkable and surprisingly readable.
Churchill, Biderman & Herrick — Magic: The Gathering is Turing Complete (free)	§2's last row , done properly — and a good illustration of how little a system needs.
Aaronson — NP-complete Problems and Physical Reality (free)	§3 and §4 assessed carefully — what physics does and does not offer computation, including every proposal in §4.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a lambda calculus interpreter** with beta reduction and capture-avoiding substitution.
- 2 Define Church numerals and addition** and verify $2 + 3$.
- 3 Define booleans and if-then-else** as lambda terms.
- 4 Implement the Y combinator** and use it to write factorial without a recursion construct.

- 5 **Relate Y to the recursion theorem** in writing.
- 6 **Implement a two-counter machine** and simulate a small Turing machine on it.
- 7 **Build a glider gun in the Game of Life** and describe how signals compose.
- 8 **Determine whether a configuration language you use is Turing-complete**, and say how you decided.
- 9 **Write a configuration that does not terminate**, if it is.
- 10 **Pick one hypercomputation proposal** and identify precisely where the infinity is assumed.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the lambda calculus's syntax and its single rule.
- 2 How are numbers, booleans, and recursion obtained?
- 3 What is the Y combinator, in Module 09's terms?
- 4 Name six Turing-complete systems and what each has.
- 5 Why is accidental Turing completeness a hazard, and what is the response?
- 6 What does quantum computation change, and what does it not?
- 7 Distinguish the Church–Turing thesis from its extended version.
- 8 Name five hypercomputation proposals and the assumption behind each.
- 9 Why is the agreement of the models the real result?